Architectural Asset Description

AAD

Azure Cosmos DB for NoSQL

AG Insurance

Version 1.0.0 - Draft

Contact:

Table of contents

Document control

1. Ownership

Subject Matter Expert		Process Owner	Department Owner
<Expert name>		Dries Verbiest	IT Architecture
Review frequency:	Yearly		
Responsible of the review:	Dries Verbiest		
Governing body responsible for the document validation	1st level	IT	
	2nd level	IT Architecture Board	
	3rd level		
	4th level		

2. Version History

Version no.	Version date	Author	Change description
1.0	08-06-2026	Reda Moukhlissi	init

Governance

Architectural Asset Description (AAD): *Architecture documentation of an Asset serving as the reference for the setup, applied principles and usage of an asset.*

Before publishing, present this AAD to ITAB for validation. Validated AADs are published by IT Architecture on the IT Repo.

Glossary

Term	Definition
AAD	Architectural Asset Description
ASG	Architectural Solution Guidance
CMK	Customer-Managed Key
PITR	Point-in-Time Restore
RU	Request Unit; Cosmos DB unit of throughput measuring CPU, memory and IOPS
RBAC	Role-Based Access Control
IaC	Infrastructure as Code
DRP	Disaster Recovery Plan
RPO	Recovery Point Objective
RTO	Recovery Time Objective
CC1 / CC2	(criticality classifications for applications)*
IIQ	Identity IQ; AG standard identity governance platform
Landing Zone	AG Azure subscription model isolating workloads per application and environment
Change Feed	Cosmos DB native mechanism that streams item-level changes in order

1 Executive summary

This document defines how Cosmos DB for NoSQL is expected to be used within AG. It sets the baseline architecture, security rules, and operational expectations that project teams must follow when deploying and running Cosmos DB in Azure.

Cosmos DB is used for specific types of workloads where low latency, horizontal scale, or event-driven patterns (via change feed) are required.

This AAD covers the NoSQL API only. MongoDB, Cassandra, Gremlin, Table and PostgreSQL-compatible APIs are out of scope.

2 Context & scope

2.1 Context

Cosmos DB for NoSQL is introduced as a managed NoSQL capability for workload that require low-latency access, flexible schema, global distribution or horizontally scalable document-oriented storage.

It is NOT the default replacement for SQL server, azure SQL Database, Databricks or the DWH

Using Cosmos DB must be justified case by case, based on the actual access pattern, expected scale, cost, and how the data can be backed up and exported if needed.

This AAD defines the standard patterns that projects must follow when requesting or implementing CosmosDB

2.2 Scope

2.2.1 In scope

- Cosmos DB account architecture for AG Azure landing zones.
- Network integration and private access model.
- IAM, RBAC and managed identity usage.
- Security baseline.
- Performance, partitioning and scalability principles.
- Backup, restore and recovery strategy.
- Sovereignty and exit considerations.
- Monitoring, alerting and operational integration.
- DRP testing expectations.
- Deployment standards.
- Non-production data refresh and scrambling principles.
- RACI

2.2.2 Out of scope

- Application data model design in detail.
- Functional schema design.
- Application code implementation.
- Full data architecture for analytical platforms.
- Replacement strategy for existing SQL workloads.
- Data replication.
- Data governance (naming convention, modulization rules…)
- GDPR execution engine.
- Everything that is not in the scope

3 Asset Identity & Classification

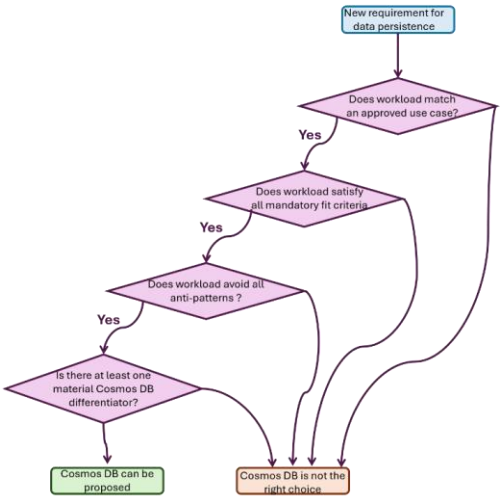
This section establishes the identity of the asset within the AG architecture landscape, providing the metadata needed for portfolio management and lifecycle governance.

Attribute	Value
Asset Name	Azure Cosmos DB for NoSQL
Asset Owner	Database Administration & Engineering
Vendor / Provider	Microsoft Azure
Service type	Fully managed PaaS
Architecture domain	
Lifecycle status	Strategic (approved for new workload)

Commented [AG1]: @Moukhlissi Reda This attribute is new for me, not yet approved by ITAB. Governance ? List of potential values ? Not clear at current time. This should be aligned with Karl proposal for technology lifecycle.

4 Approved Use Cases & Fit Criteria

This section defines when Cosmos DB is the right tool and when it is not. A workload must match at least one approved use case, satisfy all mandatory fit criteria, demonstrate a material Cosmos DB differentiator and avoid all anti-patterns



4.1 Approved Use Cases

Use case	Suitable when
High-scale key-based lookup	Very high-throughput point read/writes addressable by a well-chosen key, with predictable single-partition access. (ex : Policy lookup based on policy number)
Operational read models / denormalised projections	A denormalised, read-optimised copy of data from the main system of record. (ex: a 360 customer view on aggregated data on pre-joined document per customer)
Digital journey or workflow state	Bounded application state must be retrieved and updated independently using a known identifier.
User preferences	Application-owned preference data is stored as a bounded document and accessed primarily by a user identifier.
Globally distributed operational data	The application requires active-active writes across multiple Azure regions and can support the associated consistency model.
Heterogeneous catalogue serving	Large, read-heavy catalogue containing different entity types, each with its own attributes and limited relationships to other entities.

4.2 Mandatory Fit Criteria

The following conditions must be met in all cases. If one of them is not satisfied, Cosmos DB is usually not the right choice and another solution should be considered.

Criteria	Description
Access is predominantly key-based	The dominant access path is a point read/write addressed by partition key and item id. The workload does not depend on cross-partition queries or cross-entity joins.
No multi-partition transactions required	The workload does not require atomic transactions spanning more than one logical partition.
A sound partition key exists	A partition key can be identified that is high-cardinality, distributes throughput evenly and avoids hot partitions across the expected lifetime and growth of the workload.
Analytics are served separately (if any)	Reporting, aggregation and analytical access are not served by queries against the operational container.

Important: A document-shaped data model, a preference for JSON or schema flexibility alone is not a differentiator. Azure SQL Database and SQL Server support JSON natively. The justification must be one of scale, distribution, latency SLA, change feed or schema at volume — not the shape.

4.3 Anti-Patterns & Disqualifiers

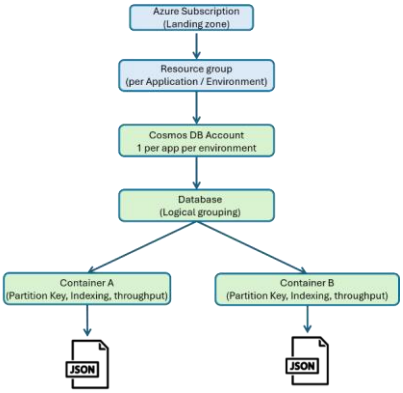
Where any of the following conditions is present, a relational database should be used even if Cosmos DB could be made to work technically.

- The data volume is small or steady. Cosmos DB is built for scale; without it, you pay for capacity you will never use. A relational database with a JSON column is simpler and cheaper.
- The data is highly connected and will be queried in unpredictable ways. Core data that links many entities through relationships that must be enforced and often needs atomic multi-entity updates.
- Cosmos DB is proposed only because the data is JSON or schema-flexible.
- The workload cannot be partitioned around a stable business boundary.

5 Technical Architecture

5.1 Resource Hierarchy

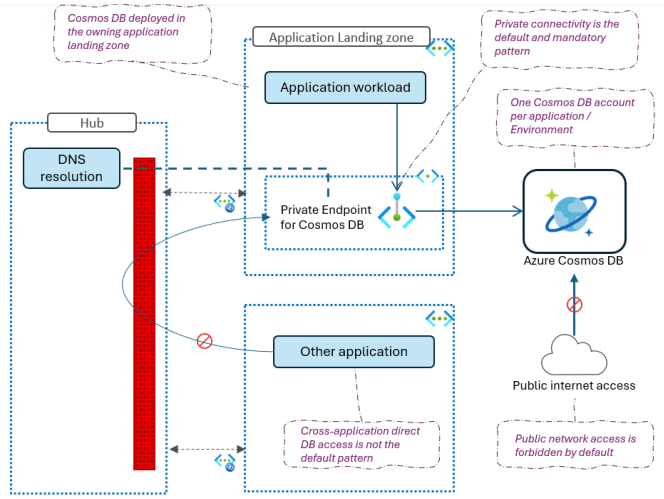
Every Cosmos DB deployment at AG follows the hierarchy below. Understanding this structure is essential for correct provisioning, cost allocation and access control



5.2 Landing Zone Placement & Network Architecture

Cosmos DB accounts are deployed within the application landing zone that owns the workload. This ensures alignment with the application lifecycle, security classification, cost ownership and operational responsibility

Principale	Rationale
One Cosmos DB account per application per environment	Provides clear ownership, isolation, cost allocation, security boundary, lifecycle management and operational accountability
Cosmos DB is deployed in the owning application landing zone	Keeps the database aligned with the application lifecycle, security classification, cost owner and operational responsibility
Private connectivity is the default and mandatory pattern	Cosmos DB must be accessed through private endpoints integrated with the AG landing zone network model.
Public network access is forbidden by default	Cosmos DB accounts must not be exposed on the public internet. Any exception requires explicit architecture and security approval
Direct cross-application access is not the default pattern	Applications should expose data through approved integration patterns such as APIs, events or data platform flows



6 Security Baseline

6.1 Design Principals

6.1.1 IAM & Access Management

Principal	Rationale
Access to Cosmos DB must follow the standard AG IAM process	Access requests, approvals, reviews and revocation must be aligned with the IIQ-based RBAC process
Control-plane and data-plane access must be separated	Managing the Cosmos DB Azure resource must not automatically grant access to business data stored in containers
Managed identity is the default access pattern for applications	Avoids unmanaged secrets for Azure-hosted workloads
Direct human access to production data should remain rare and controlled	Production data access must be justified, approved, traceable and time-bound where possible
Key-based access is not the standard pattern	Local keys or connection strings introduce higher operational and security risk and must only be used through approved exceptions.
Access must be scoped to the minimum required level	Permissions should be granted at the lowest practical scope: account, database or container depending on the use case.

6.1.2 Encryption & Key Management

Principal	Rationale
Encryption at rest is mandatory for all accounts	Protects stored data and aligns with AG baseline for managed cloud data services
Customer-managed keys must be assessed for Confidential data and are mandatory for Highly Confidential or Secret data	Provides stronger control over key ownership and supports sovereignty and regulatory requirements.
Local/key-based authentication is disabled by default where technically possible	Reduces exposure from long-lived secrets and enforces identity-based access patterns

Data Classification Rule: The classification of the Cosmos DB account must be based on the most sensitive data stored in the account. When multiple containers are hosted in the same account, the highest classification applies to the account-level security posture.

7 Performance, Partitioning & Scalability

7.1 Design Principals

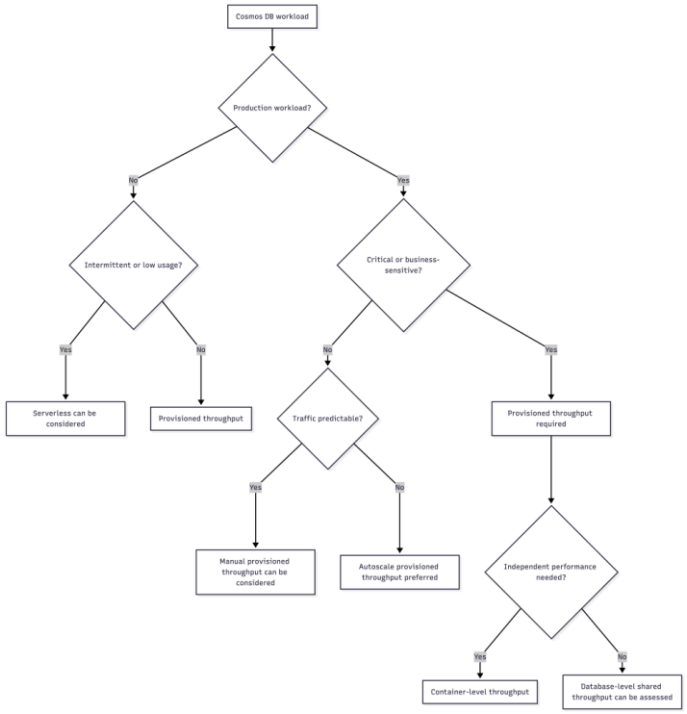
Princpale	Rationale
Performance design must be based on known access patterns	Cosmos DB only works well if the main access patterns are known upfront
The partition key is a mandatory design decision reviewed by the DAE team	Partitioning directly impacts scalability, cost, query efficiency and operational stability
If access patterns are unclear or keep changing, Cosmos DB is not a good fit	Such workloads are usually better suited for relational databases or the data platform.
RU sizing and throughput mode must be estimated before production approval	Capacity and cost must be understood early enough to avoid unexpected production cost or throttling.
Indexing policy must be reviewed for write-heavy, cost-sensitive or query-intensive workloads	The default indexing policy may not be optimal for every workload.

7.2 Throughput and RU Sizing

Cosmos DB throughput configuration must be selected during solution design. The objective is to provide enough capacity for the application while keeping cost predictable and aligned with the workload criticality.

Configuration	Reference architecture position
Autoscale provisioned throughput	default recommendation for production workloads with variable or uncertain traffic patterns
Manual provisioned throughput	suitable for stable workloads with predictable and measured consumption
Serverless	Allowed for dev/test, proof of concept, prototypes or low/intermittent workloads. not the default for critical production workloads
container-level throughput	Preferred for critical containers or containers with specific performance, isolation or cost requirements
database-level shared throughput	allowed for small related containers with low/medium traffic and no strict isolation requirement

Selection Logic :



8 Integration Patterns

This section describes how applications should interact with Cosmos DB and how data should be exposed or shared.

8.1 Application Access

- Applications access Cosmos DB through the Azure Cosmos DB SDK (preferred) or the REST API via private endpoints.
- Managed identity with data-plane RBAC is the standard authentication method.
- Connection must use direct mode (TCP) for production workloads to minimize latency.
- Retry policies must be configured in the application to handle transient failures and throttling

8.2 Change feed Integration

The Cosmos DB change feed provides a persistent, ordered log of item-level changes and is the preferred mechanism for event-driven integration.

- Use the change feed processor library (SDK) or Azure Functions Cosmos DB trigger.
- Change feed consumers must handle idempotency, as events may be delivered more than once.
- Change feed does not capture deletes by default; use soft delete with a TTL-based purge if delete propagation is needed.

8.3 Cross-Application Data Access

Direct cross-application access to Cosmos DB is not the default pattern. When data must be shared:

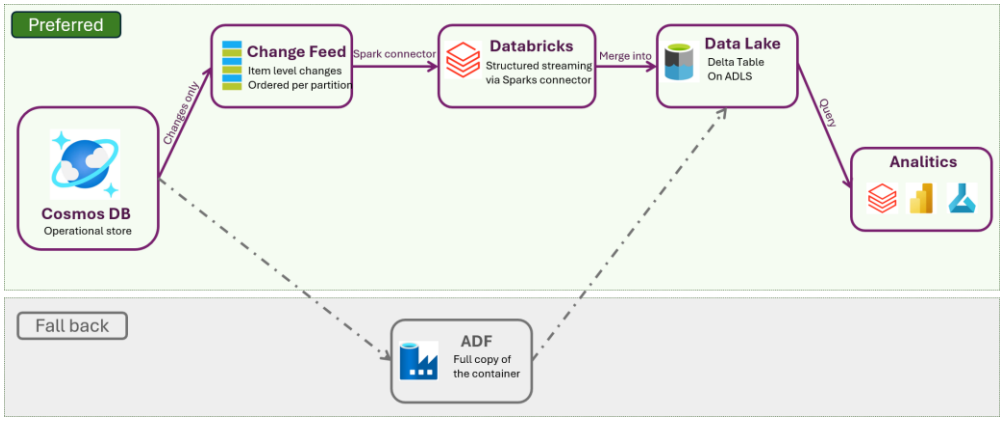
- Expose data through APIs owned by the data-owning application.
- Use event-based integration (change feed to Event Hub / Service Bus) for asynchronous propagation.
- For analytical consumption, push data to the data platform through approved data flows.

8.4 Data platform integration

Cosmos DB should not be used directly for reporting or analytics. Data needed for those use cases must be pushed to the data platform.

Preferred Pattern : Change feed via Databricks Structured Streaming (incremental)

The recommended integration path uses the Cosmos DB Spark connector in Databricks to read the change feed and merge changes into a Delta table on ADLS. This approach reads only the changes, so the RU consumption depends on how much data is actually changing, not the full dataset size.



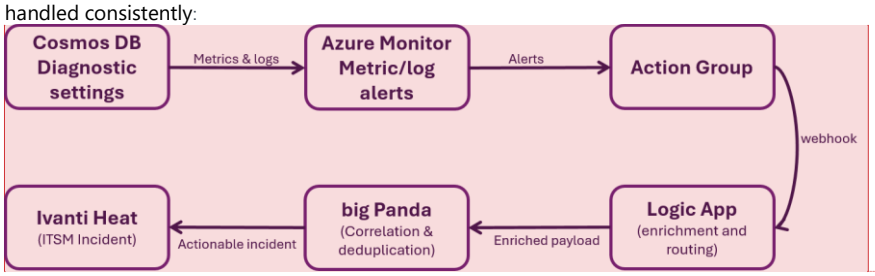
For small containers or one-time loads, ADF Copy Activity can export the full container intto a Delta table. This pattern is simpler to set up but must not be used as the standard for large or frequently updated containers.

Rational : a full export reads every document regardless of whether it changed, consuming RUs proportional to the total data volume and competing with the operational workload for throughput.

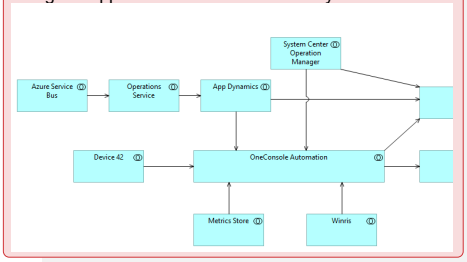
9 Operational Model

9.1 Monitoring and Alerting

Monitoring must follow the standard Azure Monitor setup used across AG, so alerts are routed and handled consistently:



Commented [AG2]: @Moukhlissi Reda What about using the application name as defined by Architecture ?



Principe	Rationale
Cosmos DB monitoring must follow the AG Azure Monitor reference architecture	Ensures consistent alert routing, incident creation and operational governance across Azure workloads
Actionable Cosmos DB alerts must be routed through the standard alerting chain	Provides centralized alert orchestration, incident correlation and Heat integration
Alert payloads must use the standard schema and be enriched with operational metadata	Improves routing, correlation, ownership identification and incident handling
Only actionable alerts must be forwarded to BigPanda / HEAT	Reduces alert fatigue and avoids non-relevant tickets

Cosmos DB alert ownership must be explicitly defined	Each monitored Cosmos DB account must have a clear application/service owner accountable for alert quality and response
Alerting must be validated before production onboarding	A Cosmos DB workload is not production-ready until the core alerts, routing and ownership are validated

Some examples of alerts to monitor (requirements may change from one application to another) :

P1 (Critical) : Availability Drop, Server Error (5xx), Throttling, RU saturation

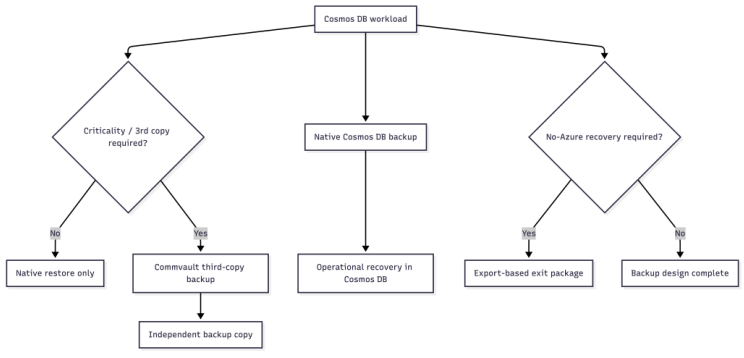
P2 (Warning) : RU consumption rising, Latency degradation, Storage growth, Hot partition detected, change feed lag

P3 (Informational) : Cost anomaly, configuration drift, Capacity trends

9.2 Backup and restore strategy

Native Cosmos DB backup is the baseline for recovery. Commvault is used only when a third independent copy is required. sovereign exit is not solved by backup alone and is handled through a separate export-based exit pattern explained in the next section

Princpale	Rationale
Built-in Cosmos DB backup is mandatory for production workloads	Provides the default operational recovery capability for Cosmos DB
Continuous backup / point-in-time restore is the preferred production baseline	Supports recovery from accidental deletion, data corruption or application errors
Commvault is used as third-copy backup for workloads requiring an independent backup copy (CC1 / CC2)	Aligns Cosmos DB protection with AG's enterprise backup standard for critical applications
Commvault backup is complementary to built-in Cosmos DB backup.	It does not replace the native operational restore capability
Sovereign exit recovery is addressed separately.	Backup and third copy do not automatically provide a full recovery path outside Azure



9.3 Disaster recovery

Cosmos DB provides built-in high availability through replica sets and zone redundancy (when enabled) within a region, and optional cross-region replication for disaster recovery. The deployment model must be selected during design based on the workload's criticality classification and RPO/RTO requirements

Option 1 : Single Zone, w/o zone Redundancy

- All replicas sit in ONE availability zone
- AZ failure → potential full downtime
- Region failure → unavailable
- Not acceptable for production
- May be used for non-prod environments only

Option 2 : Single Zone, with zone Redundancy

- Replicas spread across 3 availability zones
- AZ failure → no downtime
- Region failure → unavailable
- Not acceptable for CC1/CC2 applications
- May be used for non-critical production that tolerate extended RTO during full region outage

Option 3 : Multi region (Active- Standby)

- Single write region + read replica + zone Redundancy
- AZ failure → no downtime
- Region failure → auto-failover
- Mandatory for CC1/CC2 production workload
- Consider PPAF for sub 3-min RTO

Option 4 : Multi region Write (Active-Active)

- Multi region write
- Contraints : No strong consistency, conflict resolution in app, full RU in every region
- Region failure → RPO>0, RTO =0
- Not recommended : complexity overweight the RTO benefit. Exception only with architecture approval.

9.4 Non Production Data management

This section defines the reference approach for refreshing non-production Cosmos DB accounts with scrambled production data (this section still need to be reviewed and validated by Database Administration Engineering team)

Unlike relational databases where a backup can be restored and scrambled in-place using stored procedures, Cosmos DB requires an export-transform-import pipeline because the database has no server-side procedural language capable of iterating across partitions and applying data masking at scale

Use case	Suitable when
PII must be masked before data is made available in qualification, development or any environment where broad access is granted	GDPR and AG data protection policies require that personal data is not exposed to teams that do not have a legitimate need to access it in clear text
The acceptance environment may receive an unscrambled copy of production data, provided access is restricted to the same level as production and the data is handled under the same classification controls	Acceptance testing sometimes requires exact production data to validate business logic, migration correctness or production-issue reproduction. Scrambling would invalidate these tests.
Each Cosmos DB workload must declare its PII-bearing JSON paths at onboarding. This declaration feeds the scrambling function and is maintained in the AG enterprise data catalog	Cosmos DB documents have flexible schemas. Unlike relational databases where columns are known and fixed, PII can live in nested objects, arrays or vary across document types. The platform cannot discover PII automatically
The same masking functions used for relational database scrambling are applied to Cosmos DB data. Only the addressing mechanism changes: JSON paths instead of table.column references	Ensures consistent data protection standards across relational and NoSQL assets

Operation considerations:

RU cost of export: The export step consumes RUs on the production account. Cost can be estimated based on the data volume

RU cost of import: The import step writes to the non-production account. Use bulk import mode and temporarily scale up the non-prod throughput for the duration of the import.

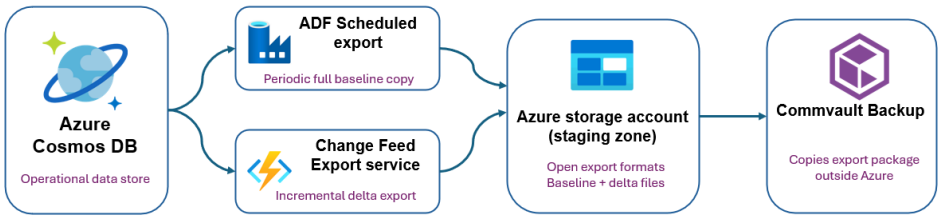
Staging data retention: Scrambled and unscrambled JSONL files in the staging storage account must be deleted after successful import.

10 Sovereign Exit Plan

Sovereign exit relies on being able to export and reuse the data outside Cosmos DB, not on Cosmos DB-native restore alone.

Native Cosmos DB backup remains the operational restore mechanism. Commvault third-copy backup provides an AG-controlled backup layer when required. If data needs to be recovered outside Azure, an additional export must exist in a format that can be restored without Cosmos DB.

The Azure Storage Account is used as a staging zone for the export package. It is not the sovereign recovery location. The recovery source for a no-Azure scenario is the Commvault-protected copy stored under AG control.



Principale	Rationale
every Project using Cosmos DB workload must assess and define its sovreign exist requirement during the design phase	Ensures the decision is concious and documented, not an oversight discovered during crisis
Exit Artifacts must use open source formats (JSONL, Parquet) that can be ingested by any database or query platform other than Cosmos DB	Depending on Azure-native restore for Exit contradict the goal of sovreignity
Not all workload require full application recovery outside Azure. The exist level (Data-only, read-only, or full recovery) must be aligned with the workload's criticality classification	avoid over-engineering exit for all applications

Required project design decisions:

Each project/application requiring sovereign exit must document the following point. This will help determine the best approach for the export

Decision	Expected answer
Exit requirement	Required / not required, with rationale
Recovery level	Data-only, read-only or full application recovery
Recovery target	Target platform or target recovery mode

Export scope	Account, database, container or selected data scope
Export approach	Baseline and incremental export principle
Data format	JSONL, Parquet or other approved format
Delete handling	Soft delete, tombstone or approved alternative
Exit RPO	Based on the last export batch copied outside Azure
Restore validation	How the recovery package will be restored and tested

11 Governance & Lifecycle

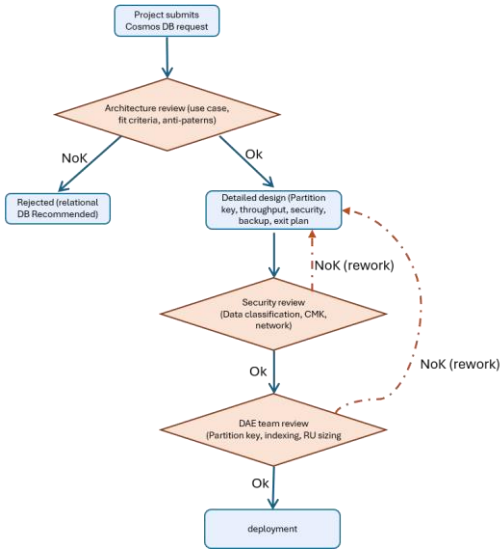
11.1 Provisioning & Deployment

Cosmos DB has several configuration decisions that are immutable or expensive to change after account creation. Unlike most Azure PaaS resources where misconfiguration can be corrected in-place, a wrong choice on Cosmos DB often means recreating the account and migrating data. The provisioning process must front-load these decisions.

Principale	Rationale
Zone redundancy, partition key, account-level API and backup mode cannot be changed after creation. These must be reviewed and confirmed before the first deployment	Correcting these post-deployment requires creating a new account and migrating all data, which is disruptive, costly and risky for production workloads
No Cosmos DB account may be deployed to production without passing the architecture review gate, which validates use case fit, partition key design, security baseline, backup strategy and exit assessment.	Prevents workloads that don't belong in Cosmos DB from being deployed, and catches partition key issues before they become production problems.
The partition key must be reviewed and approved by the DBA team independently of the architecture review. This is not a formality; it is a technical validation of cardinality, distribution and access pattern alignment	A bad partition key is the most common root cause of performance degradation, hot partitions and cost overruns in Cosmos DB. It cannot be changed without a full data migration.

Provisioning process :

New Cosmos DB deployments must follow a structured provisioning process to ensure architectural compliance, security validation and operational readiness.



11.2 RACI Management

Activity	App / Service Owner	IT Architecture	CCOE	Cloud Ops	Database Team	Cyber-security	1st Line	QM / Change
DESIGN								
Use case & fit validation	R	A	C	I	C	I		
Partition key & data model review	R	C	I	I	A			
Security baseline & network design	R	C	C	C	I	A		
Backup, exit strategy & DRP design	R	A	C	C	C	I		
IaC template development	R	C	A	C	C	I		
OPERATE								
Monitoring, alerting & dashboard setup	R	I	C	A	C	I		
Alert triage & response	C	I	I	A	R	I	C	
RU right-sizing & cost review	R/A	I	C	C	C	I		
Resource housekeeping (decommission)	A	I	R	C	I			
BACKUP & RECOVERY								
Native backup (PITR) configuration	R	C	I	C	A	I		

Commvault third-copy (if applicable)	A	I	I	C	R	I		
Restore & DRP testing	R	I	I	C	A	I		
MAINTENANCE								
Deploy changes via CI/CD	R/A	I	C	C	I			C
Non-prod refresh & scrambling	A	I	I	C	R	I		C
Decommissioning	A	C	R	C	C	I		C
INCIDENT & CHANGE								
Incident logging & routing	C	I	I	C	C		R/A	
Incident troubleshooting (data-plane)	C	I	I	A	R	I		
Raise change request & approval	R/A	C	I	C	C			A

11.3 Data Schema Governance

Unlike relational databases where the schema is enforced by Erwin, Cosmos DB does not enforce a document schema at the database level. Documents within the same container can have different structures. This flexibility makes development easier, but also means that without documentation, it becomes unclear what data exists, where PII is stored, and how it evolves

To address this, each Cosmos DB workload must maintain a **JSON Schema contract**

The JSON schema should describe the expected structure: fields, types, required vs optional, and nested objects. it serves as the contract between the application team and the platform team.

- The JSON Schema is descriptive, not enforced by the database
- The JSON Schema serves as the source for the PII field registry used in the scrambling pipeline